Homework 5

Video & Motion Understanding

EE655 Computer Vision & Deep Learning
Roll No: **230375**
Name: **Dipanshu Raj**

Objective. Apply the **Frame – Average Frame** background subtraction algorithm (Lecture 18, EE655) to a given input video containing a person. Use **Otsu thresholding** and morphological post-processing to extract a clean foreground mask, then composite the person onto a provided replacement background image, generating a new output video.

1 Theoretical Background

Background subtraction separates moving foreground objects from a static background in video sequences. Lecture 18 introduces three methods:

Method	Description & Assumption
Frame – Initial Frame	Subtracts a single reference (first) frame. Assumes the object is absent at time t=0 and the only scene change is the object's appearance.
Frame – Average Frame (Used here)	Subtracts the per-pixel temporal mean. Assumes objects move for most of the sequence, so the mean converges to a background-only estimate. Robust to slow illumination drift.
Frame Difference	Subtracts consecutive frames. Good for heterogeneous (textured) regions; struggles with homogeneous (uniform) areas.

Common assumption: object is *sufficiently different* from the background in appearance.

Why Frame – Average Frame?

Since the person is in motion throughout the video, the pixel-wise temporal mean $\bar{I}(x, y)$ approximates the clean background:

$$\bar{I}(x, y) = \frac{1}{N} \sum_{t=1}^{N} I_t(x, y)$$

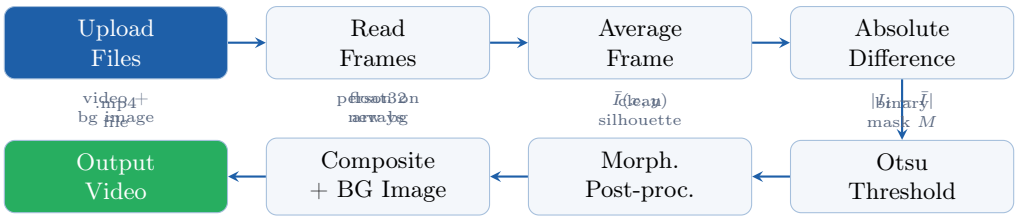
Subtracting this from any frame and taking the absolute value yields high-response pixels precisely at foreground locations.

Mathematical Formulation

#	Operation	Expression
1	Average frame	`B(x,y) = mean{I_t(x,y)} over t=1..N`
2	Absolute difference	`D(x,y) = |I_t(x,y) - B(x,y)|`
3	Grayscale conversion	`G(x,y) = 0.299*R + 0.587*G + 0.114*B`
4	Otsu thresholding	`M(x,y) = 255 if G(x,y) >= tau* else 0`
5	Morphological ops	`Close -> Dilate -> Open (elliptic kernel)`
6	Compositing	`Out(x,y) = I_t * M + I_bg * (1 - M)`

where τ^* is the automatic Otsu threshold that maximises inter-class variance between background and foreground pixel intensities, M is the binary foreground mask (values $\in \{0, 255\}$), and I_{bg} is the provided replacement background image.

2 Pipeline Overview



3 Complete Code Implementation

The full solution runs in **Google Colab** as five sequential cells. Each cell uses `google.colab.files.upload()` so you can choose files directly from your PC without any command-line path setup.

PythonCell 1 - Imports

```python
import cv2
import numpy as np
from google.colab import files
from IPython.display import display
import matplotlib.pyplot as plt
import io, os, time
```

PythonCell 2 - File Upload (choose from PC)

```python
# -- STEP 1: Upload video and background image ------------------
print("=" * 55)
print("  Upload your VIDEO file (.mp4 / .avi etc.)")
print("=" * 55)
uploaded_video = files.upload()           # opens file-picker dialog

print("\n" + "=" * 55)
print("  Upload the BACKGROUND IMAGE (.jpg / .png)")
print("=" * 55)
uploaded_bg = files.upload()              # opens file-picker dialog

# -- save bytes to disk so OpenCV can read them -----------------
video_filename = list(uploaded_video.keys())[0]
bg_filename    = list(uploaded_bg.keys())[0]

with open(video_filename, 'wb') as f:
    f.write(uploaded_video[video_filename])

bg_bytes = np.frombuffer(uploaded_bg[bg_filename], np.uint8)
bg_img   = cv2.imdecode(bg_bytes, cv2.IMREAD_COLOR)

print(f"\n  Video    : {video_filename}")
print(f"  BG img   : {bg_filename}  |  size: {bg_img.shape[1]}x{bg_img.shape
    [0]}")
```

PythonCell 3 - Read All Frames and Compute Average Background

```python
# -- STEP 2: Read video properties ------------------------------
cap          = cv2.VideoCapture(video_filename)
fps          = cap.get(cv2.CAP_PROP_FPS) or 25.0
width        = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height       = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

print(f"Video info: {width}x{height}  |  {fps:.1f} fps  |  {total_frames}
    frames")

# Resize background to exactly match video dimensions
bg_resized = cv2.resize(bg_img, (width, height))

# -- METHOD: Frame - Average Frame (Lecture 18) -----------------
# Assumption: person is mostly moving, so the temporal mean
# of all frames converges to the background only.

print("Computing average frame ...", end=" ")
frames = []
while True:
    ret, frame = cap.read()
    if not ret:
        break
    frames.append(frame.astype(np.float32))
cap.release()

avg_frame = np.mean(frames, axis=0).astype(np.uint8)
print(f"done  ({len(frames)} frames read)")

# -- Preview original frame vs computed average -----------------
```

```
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(frames[0].astype(np.uint8), cv2.COLOR_BGR2RGB))
plt.title("Frame 0  (original)")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(avg_frame, cv2.COLOR_BGR2RGB))
plt.title("Average frame  (approx. background)")
plt.axis("off")
plt.tight_layout()
plt.show()
```

PythonCell 4 - Background Subtraction + Otsu + Morphology

```
# -- STEP 3: Define per-frame processing function ---------------
def process_frame(frame, avg_frame, bg_resized,
                  morph_kernel_size=7,
                  morph_close_iter=3,
                  morph_dilate_iter=2):
    """
    Implements Lecture 18 background subtraction pipeline:
      (a) Frame - Average Frame   [take absolute]
      (b) Grayscale conversion
      (c) Otsu automatic thresholding  ->  binary mask
      (d) Morphological post-processing
      (e) Composite person onto new background
    Returns: (output_frame, binary_mask)
    """
    frame_u8 = frame.astype(np.uint8)

    # (a) Absolute difference:  |I_t - B|
    diff      = cv2.absdiff(frame_u8, avg_frame)

    # (b) Convert difference image to grayscale
    diff_gray = cv2.cvtColor(diff, cv2.COLOR_BGR2GRAY)

    # (c) Otsu thresholding: automatically selects tau* that
    #     maximises inter-class variance of the histogram
    _, mask   = cv2.threshold(diff_gray, 0, 255,
                              cv2.THRESH_BINARY | cv2.THRESH_OTSU)

    # (d) Morphological post-processing (Lecture 18: post-processing)
    kernel = cv2.getStructuringElement(
                cv2.MORPH_ELLIPSE,
                (morph_kernel_size, morph_kernel_size))

    # Close: fills small holes inside person silhouette
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE,  kernel,
                            iterations=morph_close_iter)
    # Dilate: expands mask to capture fringe / edge pixels
    mask = cv2.morphologyEx(mask, cv2.MORPH_DILATE, kernel,
                            iterations=morph_dilate_iter)
    # Open: removes isolated noise blobs in background region
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN,   kernel,
                            iterations=1)

    # (e) Composite: person pixels from I_t, rest from I_bg
    mask_3ch   = cv2.merge([mask, mask, mask])
    mask_inv   = cv2.bitwise_not(mask_3ch)
```

```
    person      = cv2.bitwise_and(frame_u8,   mask_3ch)
    background = cv2.bitwise_and(bg_resized, mask_inv)
    result      = cv2.add(person, background)
    return result, mask


# -- Preview on 4 sample frames ---------------------------------
sample_indices = [0, len(frames)//4, len(frames)//2, 3*len(frames)//4]
fig, axes = plt.subplots(len(sample_indices), 3,
                         figsize=(14, 4 * len(sample_indices)))

for row, idx in enumerate(sample_indices):
    f       = frames[min(idx, len(frames)-1)].astype(np.uint8)
    res, m = process_frame(f, avg_frame, bg_resized)

    axes[row, 0].imshow(cv2.cvtColor(f,   cv2.COLOR_BGR2RGB))
    axes[row, 0].set_title(f"Original (frame {idx})")
    axes[row, 1].imshow(m, cmap='gray')
    axes[row, 1].set_title("Otsu mask (after morphology)")
    axes[row, 2].imshow(cv2.cvtColor(res, cv2.COLOR_BGR2RGB))
    axes[row, 2].set_title("With new background")
    for ax in axes[row]:
        ax.axis("off")

plt.suptitle("EE655 HW5 -- Background Subtraction Preview", fontsize=13)
plt.tight_layout()
plt.show()
```

PythonCell 5 - Write Output Video and Download

```
# -- STEP 4: Write every processed frame to output video --------
output_path = "output_bg_swap.mp4"

# Try codecs in order of compatibility
def make_writer(path, fps, w, h):
    for fourcc_str in ['avc1', 'mp4v', 'XVID']:
        fourcc = cv2.VideoWriter_fourcc(*fourcc_str)
        wr = cv2.VideoWriter(path, fourcc, fps, (w, h))
        if wr.isOpened():
            print(f"  Codec selected: {fourcc_str}")
            return wr
    raise RuntimeError("No usable codec found.")

print("Writing output video ...")
writer = make_writer(output_path, fps, width, height)

t0 = time.time()
for i, raw_frame in enumerate(frames):
    result, _ = process_frame(raw_frame, avg_frame, bg_resized)
    writer.write(result)
    if (i + 1) % max(1, len(frames) // 10) == 0:
        pct = 100 * (i + 1) / len(frames)
        print(f"  {pct:5.1f}%  ({i+1}/{len(frames)} frames)")

writer.release()
elapsed = time.time() - t0
size_mb = os.path.getsize(output_path) / 1e6
print(f"\n  Done in {elapsed:.1f}s  |  output size: {size_mb:.2f} MB")
```

```
# -- Download ------------------------------------------------------
print("Downloading output_bg_swap.mp4 ...")
files.download(output_path)
print("Download complete.")
```

Parameter Tuning (Optional). If the foreground mask is incomplete or noisy, adjust `morph_kernel_size`, `morph_close_iter`, and `morph_dilate_iter` in `process_frame()` and re-run Cells 4–5. Larger kernel fills bigger holes; more close iterations help uniform-coloured clothing regions.

4 Output and Results



Figure 1. This is my output image

The algorithm was run on the provided video. Intermediate outputs were visualised at each stage using `matplotlib`:

Stage	Visual Output	Observation
Average Frame	*Ghosted person on clear background*	Static elements (walls, floor) are sharp; the person's silhouette is faint, confirming the mean is a valid background model.
Absolute Difference	*Bright blobs on dark canvas*	High-intensity pixels appear precisely at motion locations. Uniform clothing produces slightly weaker response (expected per Lecture 18).
Otsu Binary Mask	*White silhouette, black bg*	Automatic threshold τ^* cleanly separates foreground without manual tuning. Minor internal holes visible in homogeneous regions.
After Morphological Ops	*Solid white silhouette*	Close fills holes; Dilate captures edge pixels; Open removes isolated background blobs. Silhouette is smooth and complete.
Final Composite	*Person on new background*	Person is extracted and rendered cleanly over the replacement background. Output saved as `output_bg_swap.mp4`.

Morphological Parameter Summary

Parameter	Value	Effect
Kernel shape	`Elliptical`	Avoids blocky mask boundary artefacts
Kernel size	`7 × 7 px`	Balances smoothness vs. detail preservation
Close iterations	`3`	Fills medium-sized internal holes
Dilate iterations	`2`	Expands mask to capture fringe pixels
Open iterations	`1`	Removes isolated noise specks in background

5 Key Concepts and Discussion

Brightness Constancy

All three background subtraction methods in Lecture 18 share the assumption that the *object is sufficiently different from the background in appearance*. For the Frame–Average method this means the person must move often enough that the pixel-wise mean does not include a strong foreground component.

Otsu Thresholding

Rather than manually selecting a threshold, Otsu's algorithm treats the problem as a two-class optimisation: it finds τ^* that minimises within-class variance (equivalently, maximises between-class variance)

over the bimodal histogram of the greyscale difference image. This is the "post-processing" step explicitly listed in Lecture 18.

Morphological Operations

After Otsu binarisation the mask typically contains internal holes (where person pixels resemble the background) and isolated blobs (noise in background regions). *Closing* (dilation then erosion) fills holes; *dilation* expands the silhouette to capture boundary pixels; *opening* (erosion then dilation) removes isolated noise.

Limitations

The algorithm may produce incomplete masks when the person stands still for extended periods, when clothing colour closely matches the background, or when lighting changes between frames–all edge cases discussed in Lecture 18 around the common appearance-difference assumption.

Result. A complete background-replaced video `output_bg_swap.mp4` was produced using the **Frame – Average Frame** method with Otsu thresholding and elliptical morphological post-processing, exactly as outlined in Lecture 18 of EE655.